

RESUMEN IA SEGUNDO PARCIAL

BÚSQUEDA – UNIDAD 3

La búsqueda es una de las áreas fundamentales de la IA. Se trata sobre encontrar una secuencia de acciones que permita a un agente pasar de un estado inicial a un estado objetivo.

AMBIENTE

Este tipo de problemas se dan en ambientes muy específicos, pero que son los más simples para un agente inteligente:

- Totalmente observables: El agente puede ver todo lo que ocurre en su entorno
- Deterministas: Si el agente realiza una acción, siempre sabe cual será el resultado. No hay azar.
- Episódico: Las decisiones del agente en un momento no afectan las decisiones futuras. Cada episodio o decisión es independiente. (Más referido al ambiente de búsqueda, no tanto a las decisiones)
- Estático: El entorno no cambia por si mismo mientras el agente esta pensando o actuando
- Discreto: Hay un numero finito de estados y acciones posibles. No hay grados o puntos intermedios.
- Agente individual: Solo hay un agente actuando
- Conocido: El agente tiene toda la información sobre las reglas del juego, los estados y las posibles acciones.

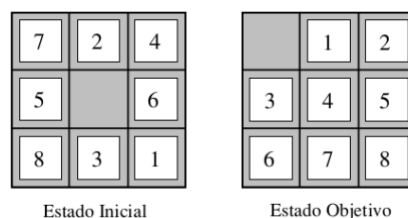
COMPONENTES

Se trata de las definiciones necesarias para construir la descripción del desafío. Son universales para cualquier problema que se quiera resolver con búsqueda.

- **Estado inicial:** Donde comienza el problema
- **Estado objetivo:** El estado o conjunto de estados a los que queremos llegar
- **Conjunto de acciones posibles:** Qué movimientos o transformaciones puede realizar el agente desde cada estado.
- **Función costo del camino:** Cuánto cuesta ir de un estado a otro. Puede ser el número de pasos, tiempo, energía. Muchas veces queremos encontrar el camino de **menor** costo.

Ejemplo de problema:

8-puzzle



Para resolver un problema de búsqueda, necesitamos definir:

- **Estados:** Todas las posibles configuraciones del tablero (cómo están ordenadas las fichas).

- **Acciones:** Deslizar una ficha hacia el espacio vacío (arriba, abajo, izquierda, derecha).
- **Función de transición (o sucesor):** Dado un estado y una acción, ¿a qué nuevo estado me lleva? (Por ejemplo, si muevo la ficha '5' hacia el espacio vacío, cómo queda el tablero).
- **Función de costo del paso:** Cuánto cuesta realizar una acción. En el 8-puzzle, cada movimiento generalmente **cuesta 1**.
- **Estado objetivo:** La configuración final deseada del tablero.

RECORRIDO DE GRAFOS

- **Nodos:** Son los estados del problema (las configuraciones del 8-puzzle)
- **Aristas (o enlaces):** Representan las acciones que conectan un estado con otro. Si se puede ir del estado A al B hay una arista o enlace entre estos.

El objetivo de la búsqueda es encontrar un camino desde el nodo inicial hasta el nodo objetivo.

Algoritmos de Búsqueda:

1. **BUSQUEDA NO INFORMADA:** Estos algoritmos no tienen información sobre que tan cerca están del objetivo. Simplemente exploran el espacio de búsqueda de forma sistemática.

- **Búsqueda en Amplitud o Primero en Amplitud (Breadth-First Search - BFS):**

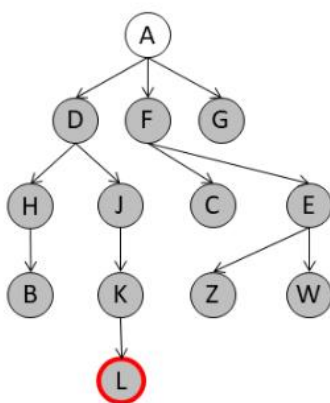
Idea: Explora el grafo nivel por nivel. Primero visita todos los vecinos del estado inicial, luego los vecinos de esos vecinos, y así sucesivamente.

Ventajas: Garantiza encontrar el camino más corto (en términos de número de pasos) si los costos de los pasos son uniformes.

Es completa (siempre encuentra una solución si existe). Garantiza encontrar el camino o solución más corto.

Desventajas: Puede **consumir mucha memoria**, ya que debe almacenar todos los nodos del nivel actual.

Ej.



m	ABIERTA
-	A
A	D,F,G
A,D	F,G,H,J
A,D,F	G,H,J,C,E
A,D,F,G	H,J,C,E
A,D,F,G,H,	J,C,E,B
A,D,F,G,H,J	C,E,B,K
A,D,F,G,H,J,C	E,B,K
A,D,F,G,H,J,C,E	B,K,Z,W
A,D,F,G,H,J,C,E,B	K,Z,W
A,D,F,G,H,J,C,E,B,K	Z,W, L

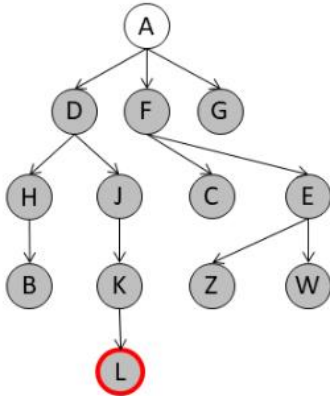
- **Búsqueda en Profundidad (Depth-First Search - DFS):**

Idea: Explora un camino lo más profundo posible antes de retroceder y probar otro camino.

Ventajas: Consume menos memoria que BFS.

Desventajas: No garantiza encontrar el camino más corto. Puede quedarse "atrapado" en un camino muy largo o infinito si no se manejan los ciclos.

Ej.



m	ABIERTA
-	A
A	D,F,G
A,D	H,J,F,G
A,D,H	B,J,F,G
A,D,H,B	J,F,G
A,D,H,B,J	K,F,G
A,D,H,B,J,K	L ,F,G

- **Búsqueda en Profundidad Limitada (Depth-Limited Search - DLS):** Es una variación de DFS que resuelve el problema de los caminos infinitos al establecer un límite de profundidad. Si llega al límite, corta esa rama. **Evita caminos infinitos.** La desventaja con este método de búsqueda es que no es completa si la solución está más allá de ese límite definido de profundidad.
- **Búsqueda Iterativa en Profundidad (Iterative Deepening Depth-First Search - IDDFS):** Combina lo mejor de BFS y DFS. Realiza DLS con límites de profundidad crecientes (0,1,2,3,...). Esto hace que sea completa, óptima y tenga ventaja de memoria sobre DFS. **La desventaja es que muchas veces puede repetir las expansiones de nodos en niveles superiores.**

2. BUSQUEDA INFORMADA: Estos algoritmos utilizan información adicional (una "**función heurística**") para estimar cuán lejos están del objetivo. Esta información les ayuda a guiar la búsqueda de manera más eficiente.

HEURÍSTICA: Es una técnica que aumenta la eficiencia del proceso, posiblemente sacrificando demandas de completitud. Es conocimiento que puede guiar el proceso de búsqueda. No garantiza la solución óptima, pero si mejoran la calidad del proceso.

El conocimiento heurístico generalmente se incorpora al proceso de búsqueda a través de una función heurística. Esta representa un grado de bondad para cada estado evaluado.

- **Busqueda Primero el Mejor:**

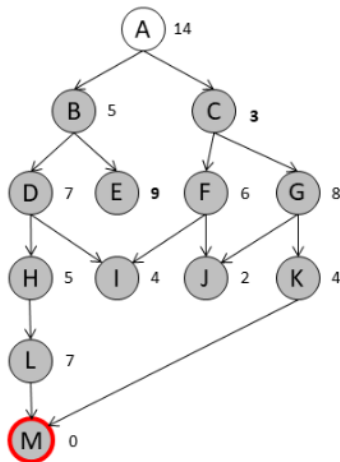
Idea: Expande el nodo que parece más cercano al objetivo, basándose solo en la función heurística.

Ventajas: A menudo es muy rápida.

Desventajas: No garantiza encontrar el camino óptimo, y puede quedar "atrapada" en un mínimo local.

- Utiliza una lista (ABIERTA) para guardar los estados a recorrer.
- La lista ABIERTA se ordena según el valor de la heurística (primero el mejor)
- Se utiliza otra lista (CERRADA) para guardar los estados recorridos.
- La heurística h' es la distancia estimada al estado objetivo.

Ej.:



m	ABIERTA	CERRADA
-	A	
A	C,B	A
C	B,F,G	A,C
B	F,D,G,E	A,C,B
F	J,I,D,G,E	A,C,B,F
J	I,D,G,E	A,C,B,F,J
I	D,G,E,	A,C,B,F,J,I
D	H,G,E	A,C,B,F,J,I,D
H	L,G,E	A,C,B,F,J,I,D,H
L	M ,G,E	A,C,B,F,J,I,D,H,L

ABIERTA (Open List / Frontier): Contiene los nodos que han sido visitados pero no expandidos (sus sucesores aún no han sido explorados). A* selecciona el nodo con el menor $f(n)$ de esta lista.

CERRADA (Closed List / Explored Set): Contiene los nodos que ya han sido expandidos. Esto evita ciclos y la reexpansión innecesaria de nodos.

○ **Busqueda A*:**

Idea: Combina el costo real para llegar al nodo actual ($g(n)$) con una estimación heurística del costo para llegar del nodo actual al objetivo ($h(n)$). **El algoritmo elige el nodo con el menor valor de $f(n)=g(n)+h(n)$.**

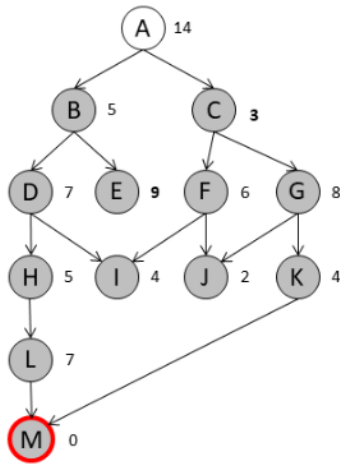
Ventajas: Es **completa** y **óptima** si la heurística es "admisible" (nunca sobreestima el costo real al objetivo) y "consistente" (también conocida como monotonicidad). Es uno de los algoritmos de búsqueda más utilizados y eficientes.

Desventajas: Puede consumir mucha memoria si el espacio de búsqueda es grande.

Heurística Admisible: Una heurística es admisible si nunca sobreestima el costo real para llegar al objetivo. Por ejemplo, en un mapa, la distancia en línea recta es una heurística admisible para la distancia real en carretera, porque nunca es mayor.

Heurística Consistente (o Monótona): Una heurística $h(n)$ es consistente si, para cada nodo n y cada sucesor n' de n , se cumple que $h(n) \leq \text{costo}(n,n') + h(n')$. Esto asegura que la estimación heurística se mantiene "lógica" a medida que avanzamos. Una heurística consistente es siempre admisible.

Ej.:



m	ABIERTA	CERRADA
-	A	
A	C,B	A
C	B,F,G	A,C
B	F,D,G,E	A,C,B
F	J,I,D,G,E	A,C,B,F
J	I,D,G,E	A,C,B,F,J
I	D,G,E	A,C,B,F,J,I
D	H,G,E	A,C,B,F,J,I
H	G,L,E	A,C,B,F,J,I,H
G	K,L,E	A,C,B,F,J,I,H,G
K	M ,L,E	A,C,B,F,J,I,H,G,K

NODO	PUNTAJE
C	4
M	4
J	5
B	6
I	7
K	7
F	8
H	8
D	9
G	10
E	11
L	11
A	14

CONCEPTOS GENERALES

Espacio de Estados (o Espacio de Búsqueda)

- El espacio de estados es el conjunto de absolutamente todos los lugares a los que se podría llegar desde el estado inicial, siguiendo las acciones posibles
- "Existe solo como una posibilidad, no está disponible ni se lo conoce":** Esto es crucial. Cuando se empieza a resolver un problema de búsqueda, no se tiene una lista completa de *todos* los estados posibles o de *todas* las conexiones entre ellos. El algoritmo de búsqueda va "descubriendo" este espacio a medida que explora. Es como si estuvieras en un laberinto gigante y oscuro; no ves todo el laberinto, solo lo que está justo en frente a medida que avanzas. El espacio de estados es el laberinto completo, exista o no lo veas.
- Camino en el espacio de estados:** Es una secuencia de lugares visitados donde cada lugar se conecta al siguiente por una **acción válida**. Si estás en el "estado A", realizas la "acción X" y llegas al "estado B", esa es una **parte de un camino**.

Salida de un Método de Búsqueda

Generalmente, la salida de un método de búsqueda es el camino de la solución. Lo más común es que, cuando un algoritmo de búsqueda termina, te devuelva la secuencia de acciones (o de estados) que te llevan del estado de inicio al estado objetivo. Por ejemplo, en el 8-puzzle, la salida sería: "mover la ficha 7 hacia abajo, luego la 5 a la izquierda, etc."

En algunos casos, la salida es el estado objetivo en sí mismo. Hay problemas donde lo único que te interesa es la **configuración final** que satisface el objetivo, no necesariamente cómo llegaste ahí.

- **Cinemática Inversa:** Imagina un brazo robótico. Le decís: "pone la mano acá". Al robot no le importa cómo llegó ahí, solo que su mano termine en esa posición final específica. El "estado objetivo" es la configuración de sus articulaciones para que la mano esté en el lugar deseado.
- **8 Reinas:** El problema de colocar 8 reinas en un tablero de ajedrez sin que se ataquen. Hay muchas configuraciones posibles que resuelven el problema. Una vez que encuentras *una* configuración válida, ese es el objetivo, y no necesariamente te interesa la secuencia de movimientos para llegar a esa configuración desde un tablero vacío (aunque podrías buscarla si fuera relevante).

Camino Solución vs. Camino de Búsqueda

Esta distinción es fundamental para entender cómo funcionan los algoritmos:

- **Camino de la Solución:** Es la respuesta final al problema. Se trata de una secuencia real y lógica de estados conectados por acciones que va directamente del estado inicial al estado objetivo. Es lo que el agente haría en el mundo real.
- **Camino de Búsqueda:** Es el rastro que deja el algoritmo mientras intenta encontrar una solución. Es la sucesión de estados que el algoritmo **exploró** (visitó y expandió), muchas veces sin una conexión lógica directa entre ellos en el mundo real. El algoritmo puede explorar un camino, darse cuenta de que no lleva a la solución, "retroceder" y explorar otro. **Estos estados no están necesariamente conectados por acciones válidas en el problema. Son simplemente los nodos que el algoritmo consideró.**
 - Ejemplo: El algoritmo puede explorar "Estado A -> Estado B -> Estado C (no lleva a nada)" y luego volver a "Estado A" y explorar "Estado A -> Estado D -> Estado E (que sí lleva a la solución)". El "camino de búsqueda" incluiría A, B, C, D, E, pero el "camino de la solución" sería solo A, D, E.

Calidad de la Solución

Medida por la función de costo del camino. Ya lo habíamos mencionado. Es cuánto "cuesta" esa secuencia de acciones (tiempo, energía, distancia, etc.).

Solución Óptima: Es simplemente la solución (el camino) que tiene el **costo más bajo posible** entre todas las soluciones válidas. Si un algoritmo es "óptimo", significa que siempre encontrará esta solución de menor costo.

RENDIMIENTO DEL MÉTODO (MÉTRICAS DE UN ALGORITMO DE BÚSQUEDA)

Estas son las cuatro "notas" principales que le ponemos a un algoritmo para saber qué tan bueno es:

COMPLETITUD: ¿El algoritmo garantiza encontrar una solución si existe? Si un algoritmo es completo, si hay un camino al objetivo, lo encontrará tarde o temprano. Si no lo es, podría fallar incluso si la solución está ahí.

OPTIMALIDAD: ¿El algoritmo garantiza encontrar la solución óptima (la de menor costo)? Si un algoritmo es óptimo, puedes estar seguro de que la solución que te da es **la mejor posible en términos de costo**.

COMPLEJIDAD TEMPORAL (Tiempo): ¿Cuánto tiempo tarda el algoritmo en encontrar la solución (o en terminar)? Se mide por el número de nodos que el algoritmo "visita" o "expande". Un algoritmo con alta complejidad temporal es lento.

COMPLEJIDAD ESPACIAL (Memoria): ¿Cuánta memoria necesita el algoritmo para encontrar la solución? Se mide por el número máximo de nodos que el algoritmo tiene que almacenar en la memoria en un momento dado. Un algoritmo con alta complejidad espacial puede agotar la memoria de tu computadora.

Conceptos Importantes relacionados con el rendimiento:

- **Factor de Ramificación (b):** Imagina que desde cada "estado" puedes tomar varias decisiones (acciones). El factor de ramificación es el número promedio de acciones posibles que puedes tomar desde un estado dado. Si desde un cruce de caminos puedes ir a 3 direcciones, el factor de ramificación en ese punto es 3. A mayor factor de ramificación, más "ramas" tiene el árbol de búsqueda y más grande es el espacio a explorar.
- **Profundidad del estado objetivo más cercano al estado inicial (d):** Es la cantidad mínima de pasos (acciones) que necesitas para ir desde el estado inicial hasta el estado objetivo (el camino óptimo). Si el objetivo está a 5 pasos del inicio, la profundidad d es 5. Estas dos variables (b y d) son cruciales para calcular las complejidades temporal y espacial de los algoritmos. Por ejemplo, en el peor de los casos, la BFS y la DFS pueden explorar $O(bd)$ nodos.

¿Estados Repetidos? Árboles vs. Grafos

Esta es una aclaración muy importante:

- **Árbol de Búsqueda:** En un árbol, cada camino desde la raíz es único. Si llegas al mismo estado por dos caminos diferentes, se considera que son **nodos diferentes en el árbol**. Los árboles no manejan estados repetidos de forma natural; si exploras un nodo, y luego llegas a la misma configuración de estado por otro camino, la tratarás **como un nuevo nodo en el árbol**.
- **Grafo de Búsqueda:** En un grafo, los nodos representan estados únicos. Si llegas al mismo estado por dos caminos diferentes, ambos caminos convergen en **el mismo nodo del grafo**. Los grafos sí manejan estados repetidos.
- **"No hacemos una búsqueda en un grafo, lo creamos 'involuntariamente' mientras buscamos":** Esto significa que, en la mayoría de los problemas de búsqueda, tú no tienes el grafo completo y dibujado de antemano. El algoritmo de búsqueda lo va construyendo dinámicamente. A medida

que expande un nodo (un estado), genera sus sucesores, y estos sucesores son nuevos nodos en tu representación mental del grafo.

- **¿Por qué es importante esto?** Si tu problema de búsqueda es realmente un *grafo* (es decir, es posible llegar al mismo estado por múltiples caminos), y no lo manejas correctamente (por ejemplo, no usando una lista de "nodos visitados" o "nodos cerrados"), tu algoritmo podría:
 - **Entrar en ciclos infinitos (si estás en un nodo A, vas a B, y de B puedes volver a A, sin fin).**
 - **Redundancia:** Perder tiempo y memoria explorando el mismo estado una y otra vez a través de diferentes caminos, incluso si ya lo visitó.
- Los algoritmos como **BFS, DFS, y A*** se pueden adaptar para funcionar en grafos (y evitar los problemas mencionados) al mantener una lista de "estados ya visitados/expandidos" (la lista CERRADA o closed_list). Antes de expandir un nodo, el algoritmo verifica si ya lo ha visto. Si ya lo ha visto y expandido, no lo vuelve a expandir, lo que ahorra tiempo y evita ciclos.

METAHEURÍSTICAS – UNIDAD 3

Las metaheurísticas son una familia de técnicas de optimización aproximada diseñadas para resolver problemas complejos. Tienen como objetivo encontrar soluciones aceptables en tiempo razonable. Se pueden entender como procedimientos con heurísticas en dos niveles, o como métodos de búsqueda de nivel superior y metodologías generales que pueden utilizarse como guías de diseño subyacentes para resolver problemas de optimización específicos.

En el diseño y funcionamiento de las metaheurísticas existen dos criterios importantes que a menudo están en conflicto:

- **DIVERSIFICACIÓN:** Capacidad de la metaheurística de explorar diferentes regiones del espacio de búsqueda, buscando nuevas soluciones y evitando quedarse atrapado en óptimos locales. Es crucial para encontrar la solución global óptima a problemas complejos.
- **INTENSIFICACIÓN:** Se enfoca en la capacidad de la metaheurística para refinar y mejorar las soluciones ya encontradas en una región específica del espacio de búsqueda. Busca explotar el conocimiento actual para mejorar las soluciones existentes.

Las metaheurísticas pueden diferir en varias características entre los métodos:

- **Inspiradas** o **no** en la naturaleza
- **Con** o **sin memoria**
- **Deterministas** (dadas las mismas condiciones iniciales y la misma secuencia de eventos siempre produce el mismo resultado) o **estocásticas** (uso de aleatoriedad en el proceso de búsqueda). Los algoritmos genéticos son estocásticos.
- **Poblacionales** (trabajan con un conjunto de soluciones) o **individuales** (trabajan con una única solución)
- Pueden tener estrategias **iterativas** (Esto significa que el algoritmo avanza paso a paso, en ciclos repetitivos (iteraciones). En cada iteración, se evalúan soluciones, se aplican operadores (como selección, cruce, mutación en algoritmos genéticos), y se generan nuevas soluciones. Este proceso se repite hasta que se cumple un criterio de parada (por ejemplo, un número máximo de

iteraciones, o una solución de calidad aceptable). Los Algoritmos Genéticos, por ejemplo, son iterativos, ya que generan nuevas "generaciones" de soluciones en cada ciclo.) o **avaras** (Una heurística o algoritmo avaro toma la mejor decisión local en cada paso con la esperanza de que esta serie de decisiones locales óptimas conduzca a una solución globalmente óptima. No se revisan decisiones anteriores y no se exploran caminos alternativos. Si bien pueden ser muy rápidos, su principal desventaja es que a menudo se quedan atrapados en óptimos locales, ya que no tienen un mecanismo para "retroceder" o explorar otras opciones una vez que se ha tomado una decisión aparentemente buena en un momento dado. Las metaheurísticas que incorporan elementos avaros suelen utilizarlos en conjunción con otros mecanismos para evitar estos óptimos locales, o para construir soluciones iniciales de buena calidad antes de aplicar la parte iterativa más exploratoria.)

Se utilizan metaheurísticas cuando no existe un método de cálculo exacto para el problema o una estrategia conocida para resolverlo. Su uso depende de la complejidad del algoritmo $O(g(n))$ y del tamaño del problema n .

ALGORITMOS GENÉTICOS

Estos algoritmos se basan en la evolución genética usando conjuntos de poblaciones que actúan de manera biológica y se reproducen. A los mismos los someten a diferentes acciones aleatorias para después realizar una selección de aquellos, que bajo un criterio, son óptimos o más adaptados. Siguiendo con el paralelismo estos algoritmos son particionados en cadenas binarias como si fueran cromosomas, genes y generaciones.

Los Algoritmos Genéticos pertenecen al grupo de los algoritmos evolutivos y están inspirados en la selección natural.

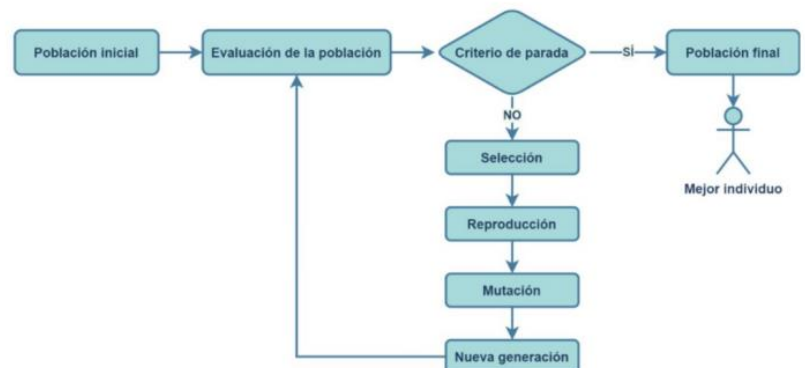
Los pasos que se llevan a cabo para inspirar estos algoritmos son:

- Selección
- Reproducción
- Mutación
- Nueva Generación

La representación de una solución en un algoritmo genético es similar a un alfabeto natural, como [A,C,G,T].

El proceso general de un algoritmo genético sigue estos pasos:

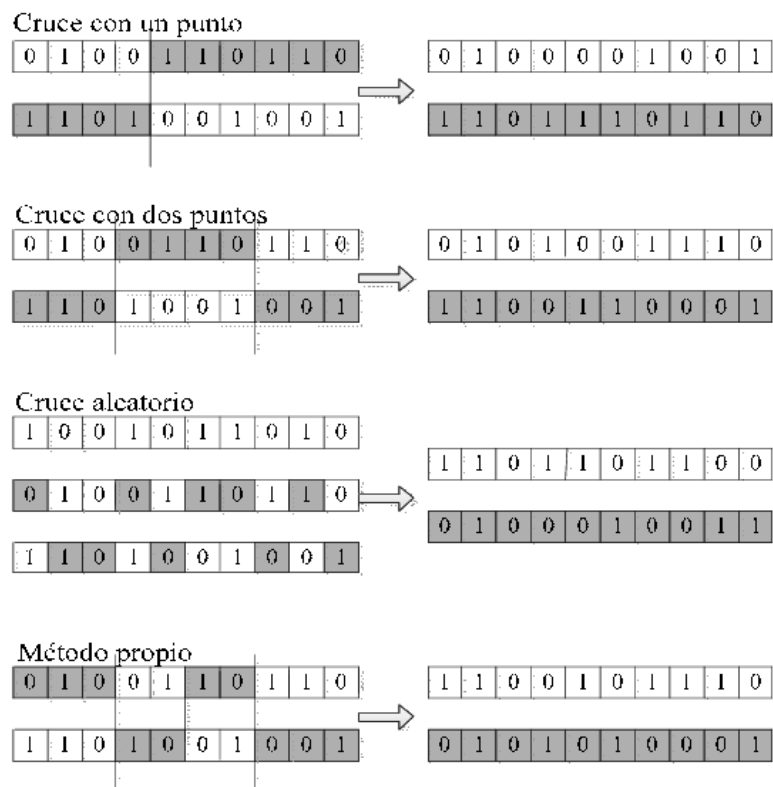
1. **Población inicial:** Se comienza con un conjunto de soluciones candidatas.
2. **Evaluación de la población:** Se evalúa la idoneidad o calidad de cada solución.
3. **Criterio de parada:** Se verifica si se ha cumplido la condición para detener la ejecución, por ejemplo, un número máximo de generaciones o una calidad de solución deseada. Si se cumple el mejor individuo de la población final es el resultado.



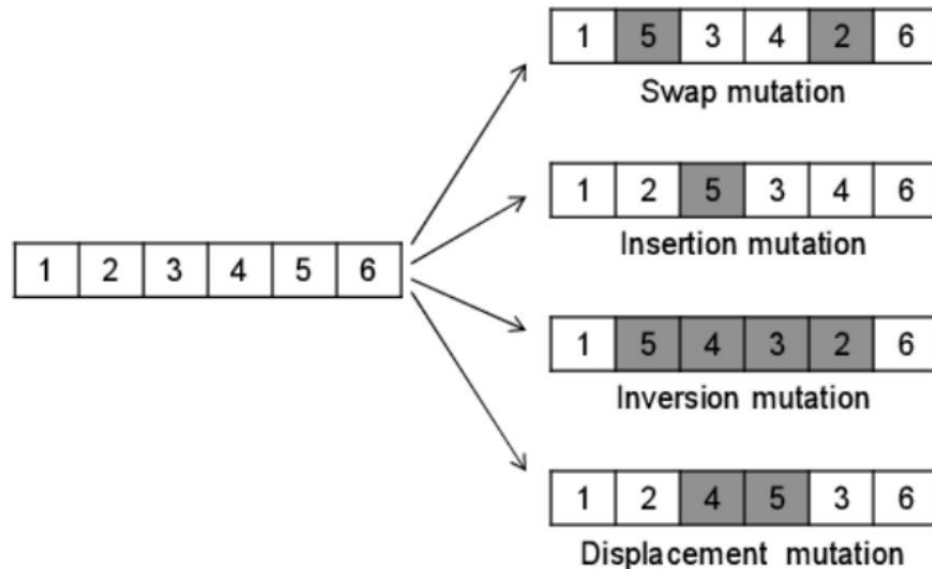
4. Si el criterio de parada no se cumple, se procede con:
 - a. **Selección:** Se eligen individuos de la población actual para reproducirse
 - b. **Reproducción:** Los individuos seleccionados crean descendencia
 - c. **Mutación:** Se introducen los cambios aleatorios en la descendencia para mantener la diversidad
 - d. **Nueva generación:** Se forma la nueva población con los individuos resultantes y el proceso se repite desde la evaluación.

Los **elementos clave** de un algoritmo genético son:

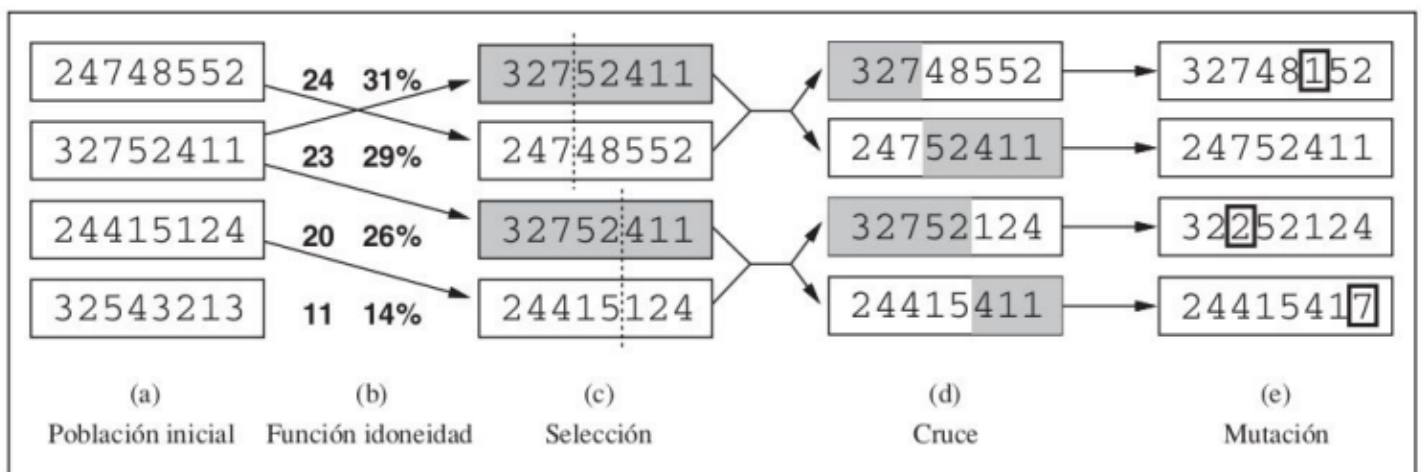
- **Representación (codificación):** La forma en que se codifica una posible solución, generalmente como una secuencia de símbolos.
- **Inicialización de la población:** Cómo se genera la primera población de soluciones.
- **Función de idoneidad (o fitness/adaptación/objetivo):** Una función que mide la calidad de cada solución, y que suele ser proporcional a la calidad de la solución, es decir que a mayor valor de la función de idoneidad, mejor es la solución obtenida por el algoritmo genético.
- **Operador de selección:** El método para elegir individuos que se reproducirán. La selección se basa en la función de idoneidad, de modo que los individuos más aptos tienen una mayor probabilidad de ser seleccionados. A menudo, es un método proporcional.
- **Operador de recombinación:** Cómo se combinan dos soluciones para crear una nueva, siendo el cruce (crossover) el más común. Imita el cruce genético, donde segmentos de los "cromosomas" de los padres se intercambian para formar descendencia. El cruce es la forma más común de este operador.



- **Operador de mutación:** Cómo se introducen pequeños cambios aleatorios en las soluciones, generalmente de forma aleatoria. Esto es vital para mantener la diversidad genética en la población y evitar que el algoritmo converja prematuramente a un óptimo local. Generalmente, es un proceso aleatorio.



- **Estrategia de reemplazo:** Cómo la nueva generación reemplaza a la antigua, comúnmente de forma generacional. Una estrategia común es la "generacional", donde toda la población anterior es reemplazada por la nueva generación
- **Criterio de corte:** La condición que determina cuándo termina el algoritmo. Puede ser, por ejemplo, alcanzar un número máximo de generaciones, encontrar una solución con una idoneidad suficientemente alta, o cuando la mejora de la población se estanca.



PASO A PASO

Inicialización

El tamaño de la población depende de la naturaleza del problema, pero normalmente contiene varios cientos o miles de posibles soluciones. A menudo, la **población inicial** se genera aleatoriamente, permitiendo toda la gama de posibles soluciones (el espacio de búsqueda).

Ocasionalmente, las soluciones pueden ser “sembradas” en áreas donde es probable encontrar soluciones óptimas.

Selección

Durante cada generación sucesiva, una parte de la población existente se selecciona para criar una nueva generación. Las soluciones individuales se seleccionan a través de un proceso basado en la **aptitud**, donde las soluciones de acondicionamiento (como medido por una función de acondicionamiento físico) son típicamente más probables de ser seleccionadas. Ciertos métodos de selección evalúan la aptitud de cada solución y preferentemente seleccionan las mejores soluciones.

Otros métodos califican solo a una muestra aleatoria de la población, ya que el proceso anterior puede llevar mucho tiempo y requiere de la creación de una buena función fitness o mérito.

La **función de fitness** se define sobre la representación gráfica y mide la calidad de la solución representada en ese tipo de problema. Es única por cada problema.

En algunos problemas es difícil o incluso imposible definir la expresión de la condición física, en estos casos, se puede utilizar una simulación para determinar el valor de la función de aptitud de un fenotipo o incluso algoritmos genéticos interactivos.

Operadores genéticos

Una vez que se ha seleccionado una parte de la población existente (basándose en su aptitud), el siguiente paso en un algoritmo genético es generar una población de segunda generación, o una nueva generación de soluciones. Esto se logra a través de la combinación de dos operadores genéticos principales:

- **Entrecruzamiento Cromosómico (Crossover):** Este operador es el análogo genético de la recombinación sexual. Para cada nueva solución que se va a producir en la siguiente generación, se selecciona un **par de soluciones "padre" de la población actual**. El crossover implica combinar material genético de estos dos padres para crear una o más soluciones "hijas". Típicamente, esto significa que partes de la codificación (cromosomas) de un padre se intercambian con partes del otro padre en un punto o varios puntos de corte. El resultado es una **nueva solución que hereda características de ambos padres**.
- **Mutación:** La mutación introduce variaciones aleatorias en las nuevas soluciones. Después de que las nuevas soluciones han sido **creadas a través del entrecruzamiento**, o incluso directamente de un solo padre, la mutación **altera aleatoriamente uno o más "genes"** (partes de la codificación) de estas soluciones. Esto puede implicar cambiar **un 0 por un 1 en una cadena binaria**, o modificar un **valor numérico dentro de un rango**. La mutación es crucial porque introduce nueva diversidad en la población, lo que ayuda a **prevenir la convergencia prematura del algoritmo a un óptimo local** y permite explorar partes del espacio de búsqueda que de otra manera no serían alcanzables a través del solo entrecruzamiento.

Al producir una nueva solución utilizando estos métodos de crossover y mutación, se crea una solución que típicamente comparte muchas características de sus padres, pero con la posibilidad de nuevas combinaciones y pequeñas alteraciones aleatorias. Esta combinación de herencia (crossover) y variación (mutación) es lo que permite a los algoritmos genéticos explorar el espacio de soluciones y evolucionar hacia mejores resultados a lo largo de las generaciones.

Aunque los métodos de reproducción que se basan en el uso de dos padres son más “biología inspirada”, algunos temas de investigación sugieren que más de dos padres puedan generar cromosomas de mayor calidad.

Estos procesos finalmente resultan en la siguiente generación de población de cromosomas, que es diferente a la generación inicial. En general, la aptitud de la solución se incrementa por este procedimiento para la población, ya que solo los mejores algoritmos de la primera generación son seleccionados para la cría, junto con una pequeña proporción de soluciones menos aptas.

Estas soluciones menos aptas aseguran la diversidad genética dentro del grupo genético de los padres y por lo tanto aseguran la diversidad genética de la siguiente generación de hijos.

Aunque el entrecruzamiento (cruce o crossover) y la mutación son considerados los principales operadores genéticos y los más comúnmente utilizados, es posible incorporar otros operadores para mejorar el rendimiento o abordar características específicas de ciertos problemas en los algoritmos genéticos.

Algunos de estos operadores adicionales pueden incluir:

- **Reagrupamiento (Reassortment):** Este operador puede implicar la reorganización de los genes dentro de un solo cromosoma o la combinación de partes de múltiples cromosomas de maneras más complejas que el cruce tradicional.
- **Colonización-Extinción:** Este mecanismo se relaciona más con estrategias de gestión de la población. La "colonización" podría implicar la introducción de nuevos individuos en la población (quizás generados aleatoriamente o a partir de una solución conocida), mientras que la "extinción" se refiere a la eliminación de individuos de baja aptitud o de regiones menos prometedoras del espacio de búsqueda para dar paso a la exploración de nuevas áreas.
- **Migración:** Comúnmente utilizado en algoritmos genéticos con poblaciones divididas en "islas" o subpoblaciones. La migración permite el intercambio de individuos entre estas subpoblaciones, lo que puede ayudar a difundir buenas soluciones a través de diferentes partes del espacio de búsqueda y a mantener la diversidad genética.

La elección y combinación de estos operadores genéticos dependerá en gran medida de la naturaleza específica del problema a resolver y del diseño particular del algoritmo genético.

Es crucial **ajustar los parámetros** de un algoritmo genético para optimizar su rendimiento en la clase de problema específico que se está abordando. Los parámetros clave a considerar incluyen la probabilidad de crossover (entrecruzamiento), la probabilidad de mutación y el tamaño de la población.

La **tasa de mutación** es un parámetro delicado:

- Una tasa de mutación muy pequeña puede llevar a la **deriva genética**. Esto significa que la diversidad genética en la población disminuye demasiado rápido, lo que puede **impedir** que el algoritmo explore nuevas áreas del espacio de búsqueda y escape de óptimos locales.
- Una tasa de mutación demasiado alta puede causar la **pérdida de buenas soluciones**. Si la mutación es demasiado frecuente o drástica, podría destruir las características beneficiosas que se han desarrollado en las soluciones prometedoras. Sin embargo, este riesgo se puede mitigar si se utiliza una **selección elitista**, donde los mejores individuos de una generación se transfieren directamente a la siguiente sin ser sujetos a los operadores genéticos.

La **tasa de recombinación (crossover)** también es vital:

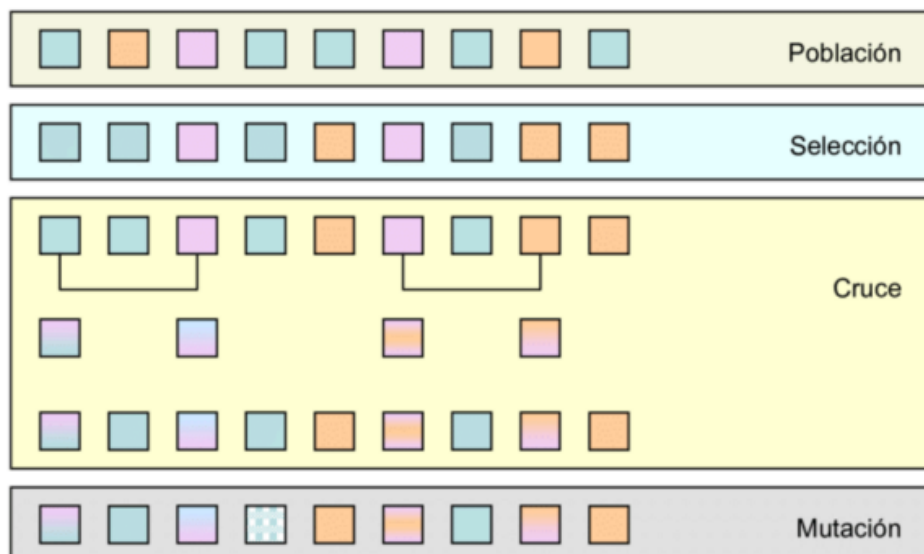
- Una tasa de recombinación muy alta puede conducir a la **convergencia prematura del algoritmo genético**. Esto ocurre cuando la población pierde rápidamente su diversidad y todos los individuos se vuelven muy similares, lo que impide una exploración efectiva del espacio de búsqueda y puede hacer que el algoritmo se estanque en un óptimo local.

Encontrar los "ajustes razonables" para estos parámetros a menudo requiere experimentación y pruebas empíricas, ya que las configuraciones óptimas pueden variar significativamente entre diferentes tipos de problemas.

Terminación

Este proceso generacional se repite hasta que se alcanza una de estas condiciones. Las condiciones más comunes para la terminación de un algoritmo genético son:

- Se encuentra una solución que satisface los criterios mínimos de calidad.
- Se alcanza un número fijado de generaciones.
- Se alcanza el presupuesto asignado, ya sea en tiempo de cálculo o dinero.
- La aptitud de la solución de la clasificación más alta (el mejor individuo) alcanza o ha alcanzado una meseta, de modo que las sucesivas iteraciones ya no producen mejores resultados.
- Inspección manual, donde un operador humano decide detener el proceso.
- Una combinación de las condiciones anteriores.



PLANIFICACIÓN – UNIDAD 3

La planificación es una rama fundamental que se encarga de diseñar secuencias de acciones (planes) para que un agente autónomo (como un robot) pueda alcanzar un objetivo específico desde un estado inicial dado.

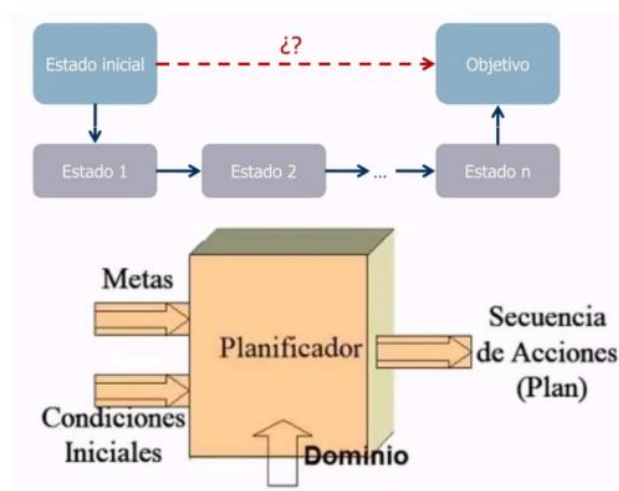
Este proceso es importante para permitir que los sistemas de IA operen de manera inteligente en el mundo real

Es el proceso de encontrar una secuencia de acciones que transforme un estado inicial en un estado objetivo.

Los **agentes planificadores** son agentes que crean el plan antes de que todo suceda, toman el estado actual (las condiciones), las acciones disponibles, los resultados posibles a esas acciones junto a las metas a largo plazo y con eso realizan **un plan de acciones a seguir**.

Estos agentes se desenvuelven en ambientes:

- **Deterministas:** Cada acción tiene un resultado predecible y único. Si un robot mueve un bloque, siempre lo moverá a un lugar esperado, sin variaciones aleatorias.
- **Completamente observables:** El agente tiene un conocimiento completo y preciso del estado actual del mundo en todo momento. No hay información oculta.
- **Finitos:** El numero de estados posibles del mundo es limitado.
- **Estáticos:** El mundo no cambia por si mismo mientras el agente no actúa.
- **Discretos:** Los estados y las acciones son distintos y separables. Hay pasos claros de un estado a otro.



Estos agentes funcionan en el contexto de la planificación automática, un área de la IA que se centra en la construcción de la secuencia de acciones.

Los problemas o agentes de planificación tienen las siguientes características:

1. **Tienen un objetivo**, posee un objetivo que desea alcanzar.
2. **Trabajan con un modelo del mundo:** Usan un modelo que describe la reacción de las acciones en el mundo y los estados posibles. Algo un poco determinista.
3. **Generar planes:** El agente busca una secuencia de acciones que le permita pasar del estado inicial o actual hacia el objetivo. Los planes pueden ser óptimos o simplemente válidos.

4. **Evaluación y ajuste:** Los agentes planificadores pueden evaluar la calidad de los planes generados y ajustarlos si es necesario, por ejemplo, si el entorno cambia o las acciones tienen otros efectos no esperados.
5. **Autonomía:** Estos agentes actúan de manera autónoma en entornos complejos, haciendo que las decisiones sean propias del mismo sin intervención humana (en lo posible)

Ejemplos:

- Sistema de navegación GPS
- Robots autónomos
- Juegos de ajedrez en IA

Se puede definir el proceso de resolución de un problema de planificación como una búsqueda en un espacio de estados donde cada estado se corresponde con una situación posible, la búsqueda comienza en una situación inicial y se lleva a cabo una secuencia de operaciones permitidas hasta alcanzar una situación objetivo.

El método A^* resuelve problemas del tipo mencionado. Es un algoritmo de búsqueda heurística que encuentra la ruta mas corta en un grafo. Aunque este algoritmo tiene la limitación de que requiere la manipulación de la descripción completa de un estado, que en contexto de problemas muy complejos se hace casi imposible de manejar) y es preferible en este escenario trabajar con partes pequeñas que hacen más eficiente la resolución del problema y luego combinar sus soluciones parciales al final. Esto lleva a la idea de la **planificación por componentes** o **planificación por pila de objetos**.

DESCOMPOSICIÓN DE PROBLEMAS

Existen dos formas importantes de descomponer:

1. **Recalculo Parcial de Estados (Representación del Cambio):** En lugar de representar y manipular la descripción *completa* de cada estado a lo largo de la búsqueda, este enfoque se centra en **los cambios** que ocurren en el estado como resultado de una acción. Aquí es donde el formalismo STRIPS (con sus listas ADD y DELETE) brilla. En lugar de almacenar Estado_completo_t y luego Estado_completo_t+1, un planificador solo necesita saber qué **predicados** se agregan y cuáles se **eliminan** cuando se aplica un operador. El estado actual se mantiene y se modifica incrementalmente.

Relación con la Planificación por Pila de Objetivos: La Planificación por Pila de Objetivos (Goal Stack Planning) utiliza inherentemente este concepto. Al aplicar un operador, solo se modifican los predicados afectados en el "estado actual" de la planificación, en lugar de generar una nueva y completa descripción de estado.

2. **División del Problema en Subproblemas (Descomposición de Objetivos):** Esta estrategia implica tomar un objetivo complejo y dividirlo en un conjunto de **subobjetivos** más pequeños y (idealmente) más fáciles de alcanzar. En lugar de intentar lograr un objetivo final de una sola vez, el planificador identifica los componentes del objetivo y trabaja en cada uno de ellos.

- o **Ejemplo:** Si el objetivo es $\text{SOBRE}(A, B) \wedge \text{SOBRE}(B, C)$, el planificador puede intentar primero lograr $\text{SOBRE}(A, B)$ y luego $\text{SOBRE}(B, C)$, o viceversa. Estos son los subobjetivos.

Relación con la Planificación por Pila de Objetivos: La Planificación por Pila de Objetivos es el ejemplo más claro de esta forma de descomposición. El algoritmo inicia con el objetivo final en la

pila, y luego, si ese objetivo no se cumple, lo "descompone" implícitamente en sus precondiciones (que se convierten en subobjetivos) y el operador necesario. Cada predicado en la pila es un subobjetivo a satisfacer.

Interacción entre ambas formas de descomposición:

Estas dos formas de descomposición **no son mutuamente excluyentes**, sino que trabajan en conjunto:

- La **división del problema en subproblemas** ayuda a estructurar el proceso de planificación a un nivel más alto.
- El **recalculo parcial de estados** (a través de los efectos ADD/DELETE de los operadores) es la técnica fundamental que permite que los subproblemas se resuelvan eficientemente sin la sobrecarga de la representación de estados completos.

La Anomalía de Sussman precisamente surge de la dificultad de **coordinar la resolución de estos subproblemas** cuando las acciones para un subobjetivo pueden interactuar negativamente (deshacer) con otro subobjetivo. Esto resalta la necesidad de planificadores que no solo dividan el problema, sino que también manejen inteligentemente las interacciones entre los subproblemas, lo que lleva a la planificación no lineal.

REPRESENTACIÓN FORMAL PARA LA PLANIFICACIÓN

Para que un sistema de IA pueda planificar, el mundo y las acciones deben ser representados de una manera estructurada que el sistema pueda manipular. Utilizan el formalismo STRIPS (STanford Research Institute Problem Solver).

Se trata de una formalización utilizada para describir como las acciones afectan al estado del mundo en un problema de planificación. Definen las acciones que un agente planificador puede realizar, junto con las condiciones necesarias y las consecuencias de dichas acciones, para ayudar a resolver problemas de planificación.

Componentes

- **PRECONDICIONES:** Son las circunstancias que deben ser verdaderas en el estado actual para que la acción pueda ser ejecutada. Si estas precondiciones no se cumple, la acción no puede realizarse (lista P)
- **EFFECTOS NEGATIVOS** (Eliminados): Son las condiciones que dejarán de ser verdaderas después de que la acción fuera ejecutada (Lista B)
- **EFFECTOS POSITIVOS** (Añadidos): Describen aspectos del estado que cambiarán cuando se realice la acción, es decir, que nuevas condiciones se vuelven verdaderas después de ejecutar la acción (Lista A).

Representación de Estados

Un estado se describe como un conjunto de predicados lógicos (también llamadas proposiciones atómicas). Estos predicados son afirmaciones sobre el mundo que pueden ser verdaderas o falsas.

Ejemplo del Mundo de Bloques:

- SOBRELAMESA(A): El bloque A está sobre la mesa.
- SOBRE(A, B): El bloque A está sobre el bloque B.
- DESPEJADO(A): No hay ningún bloque encima del bloque A.
- BRAZOLIBRE(): El brazo del robot no está sosteniendo ningún bloque.

Representación de Metas

Una meta (o estado objetivo) es también un conjunto de predicados que deben ser verdaderos en el estado final que el planificador debe alcanzar. No es necesario que todos los predicados del mundo sean verdaderos o falsos, solo aquellos especificados en la meta.

Ejemplo: $\text{SOBRE}(A, B) \wedge \text{SOBRELAMESA}(B) \wedge \text{DESPEJADO}(A)$

Queremos que A esté sobre B, B esté sobre la mesa y A esté despejado.

Operadores

Los operadores (acciones o primitivas) definen cómo el mundo cambia. Son el corazón de la planificación.

- **Nombre:** Identifica la acción (ej., APILAR, DESAPILAR, TOMAR, SOLTAR).
- **Parámetros:** Variables que representan los objetos sobre los que actúa el operador (ej., x, y para bloques).
- **Precondiciones:** Un conjunto de predicados que deben ser verdaderos en el estado actual para que el operador pueda ejecutarse. Si una precondición no se cumple, el operador no es aplicable.
- **Efectos (ADD/DELETE List):** Un conjunto de predicados que se modifican al ejecutar el operador.
 - o Lista ADD: Predicados que se vuelven verdaderos (se añaden al estado).
 - o Lista DELETE: Predicados que se vuelven falsos (se eliminan del estado).

Ejemplos de Operadores

- **APILAR(x, y):** Pone x sobre y.
 - o **P:** Precondiciones: $\text{SOSTENIENDO}(x), \text{DESPEJADO}(y)$.
 - o **A:** Efectos ADD: $\text{SOBRE}(x, y), \text{DESPEJADO}(x), \text{BRAZOLIBRE}()$.
 - o **B:** Efectos DELETE: $\text{SOSTENIENDO}(x), \text{DESPEJADO}(y)$.
- **DESAPILAR(x, y):** Quita x de y.
 - o Precondiciones: $\text{SOBRE}(x, y), \text{DESPEJADO}(x), \text{BRAZOLIBRE}()$.
 - o Efectos ADD: $\text{SOSTENIENDO}(x), \text{DESPEJADO}(y)$.
 - o Efectos DELETE: $\text{SOBRE}(x, y), \text{BRAZOLIBRE}()$.
- **TOMAR(x):** Toma x de la mesa.
 - o Precondiciones: $\text{SOBRELAMESA}(x), \text{DESPEJADO}(x), \text{BRAZOLIBRE}()$.
 - o Efectos ADD: $\text{SOSTENIENDO}(x)$.
 - o Efectos DELETE: $\text{SOBRELAMESA}(x), \text{BRAZOLIBRE}()$.
- **SOLTAR(x):** Suelta x sobre la mesa.
 - o Precondiciones: $\text{SOSTENIENDO}(x)$.
 - o Efectos ADD: $\text{SOBRELAMESA}(x), \text{BRAZOLIBRE}()$.
 - o Efectos DELETE: $\text{SOSTENIENDO}(x)$.

APILAR(x,y)
P: $\text{despejado}(y) \wedge \text{agarrado}(x)$
B: $\text{despejado}(y) \wedge \text{agarrado}(x)$
A: $\text{brazolibre} \wedge \text{sobre}(x,y)$
DESAPILAR(x,y)
P: $\text{sobre}(x,y) \wedge \text{despejado}(x) \wedge \text{brazolibre}()$
B: $\text{sobre}(x,y) \wedge \text{brazolibre}()$
A: $\text{agarrado}(x) \wedge \text{despejado}(y)$
TOMAR(x)
P: $\text{despejado}(x) \wedge \text{sobrelamesa}(x) \wedge \text{brazolibre}()$
B: $\text{sobrelamesa}(x) \wedge \text{brazolibre}()$
A: $\text{agarrado}(x)$
BAJAR(x)
P: $\text{agarrado}(x)$
B: $\text{agarrado}(x)$
A: $\text{sobrelamesa}(x) \wedge \text{brazolibre}()$

PLANIFICACIÓN POR PILA DE OBJETIVOS

Se utiliza una estructura de pila donde se insertan predicados y operadores. Cada vez que se inserta un predicado compuesto, se deben apilar además predicados atómicos.

En lugar de empezar desde el estado inicial y buscar acciones que nos lleven al objetivo (búsqueda hacia adelante), la planificación por pila de objetivos trabaja desde el **objetivo hacia atrás** hasta el estado

inicial. Se descompone el objetivo en subobjetivos, y para cada subobjetivo no satisfecho, se busca un operador que lo logre. Las precondiciones de ese operador se convierten a su vez en nuevos subobjetivos.

Los pasos a seguir son los siguientes:

1. Inicializar la pila con el estado objetivo
2. Mientras la pila no este vacía repetir:
 - Sacar el primer elemento
 - Si es un operador, aplicarlo
 - Si es un predicado verdadero, eliminarlo
 - Si es un predicado atómico falso, apilar un operador que haga que se cumpla y apilar su lista de precondiciones.
 - Si es un operador compuesto falso, volver a insertarlo.

MUNDO DE BLOQUES

Es un tipo de problema formado por una superficie plana o mesa donde existen un monton de bloques apilables y desapilables. La acción de apilar y desapilar puede ser llevada a cabo por un brazo robot que mueve los bloques de manera perfecta. El brazo solo puede levantar un bloque a la vez y el modelo completo consiste de los siguiente escenarios:

- Cada bloque puede tener los siguientes estados:
 - Sobre la mesa
 - Apilado sobre otros
 - Pieza agarrada por el brazo robot
- Tenemos una descripción de las acciones mediante 3 listas:
 - Una pila de objetivos y operadores
 - Un estado actual
 - Una pila con las acciones del plan
- Estados posibles para el bloque:
 - Despejado(X): El bloque X no tiene otro bloque sobre el
 - Brazolibre(): El brazo no tiene algún bloque agarrado
 - Sobremesa(Y): El bloque Y está sobre la mesa
 - Sobre(X,Y): El bloque X se encuentra arriba del bloque Y.
 - Agarrado(X): El bloque X se encuentra agarrado por el brazo robot.

Ej.:



Lista de estados finales:

Sobrelamesa(A)

Sobre(B,A)

Brazolibre()

Sobre(C,B)

Despejado(c)

OPERADORES/ACCIONES

TOMAR(C)

APILAR(C,B)

Lista de estado objetivo:

Despejado(C)^Sobre(C,B)^ Sobre(B,A) ^ Sobrelamesa(A)^ Brazolibre()

PASO A PASO

1. Verifico si los estados de la pila objetivo están en la pila de estados actuales, si están los elimino.
2. Si no están verifico en los predicados A que operadores me permitirían agregar esas acciones a la pila de estados actuales.
3. Para aplicar la acción a la pila de estado objetivo debo aplicarla con sus precondiciones.
4. Aplico la acción a la pila de estado objetivo.
5. Una vez que aplico la acción/operador a la pila o estado objetivo, esto repercute en la pila de estado actual eliminando y agregando estados.
6. Sigo iterativamente hasta llegar al estado objetivo en la pila de estados actuales.

AGENTES LÓGICOS Y SISTEMAS EXPERTOS – UNIDAD 4

Los agentes lógicos son aquellos que interactúan con el contexto que tienen a su alrededor. Son un tipo particular de sistemas de inteligencia artificial que se utilizan en ambientes deterministas y parcialmente observables.

Para su funcionamiento es importante comprender dos conceptos:

1. **REPRESENTACIÓN DEL CONOCIMIENTO:** El conocimiento se representa sobre el entorno o **base de conocimientos (BC)** usando algún tipo de lógica formal.

Se puede usar lógica proposicional o la lógica de predicados para intentar representar esto en un lenguaje más formal.

La BC es el componente principal de un agente basado en conocimiento, contiene hechos (lo que el agente sabe) y reglas lógicas que describen la interacción entre los mismos.

Una BC es un **conjunto de sentencias**, donde cada una de ellas es una afirmación sobre el mundo. El agente cuenta con mecanismos para añadir nuevas sentencias a su base de conocimiento y para preguntar qué se sabe en ella.

2. **RAZONAMIENTO**

El agente realiza inferencias a partir de su base mediante la utilización de reglas lógicas. No solo conecta los hechos actuales, sino que realiza deducciones sobre las nuevas.

Cada vez que el programa del agente se activa, le informa a la BC lo que ha percibido (a esto se le llama **Percepción y Actualización**) y luego consulta la BC para determinar qué acción debe ejecutar. Para tomar esta decisión, el agente debe realizar un proceso de razonamiento (**Acciones basadas en lógica**). Elige sus acciones basándose en un razonamiento profundo, buscando determinar cual es la mejor acción posible a partir de lo que sabe y puede deducir.

SISTEMA EXPERTO

Los agentes lógicos están directamente relacionados con los Sistemas Expertos (SE).

Los **Sistemas Expertos** son sistemas de IA que resuelven problemas que, bajo circunstancias normales, serían resueltos por expertos humanos. Logran esto mediante la aplicación de conocimiento específico de un dominio.

En otras palabras, son sistemas que están diseñados para simular conocimientos y el razonamiento de un experto en un campo específico. Funcionan mediante el uso de una base conocimientos que contiene hechos y reglas, junto con un motor de inferencia con el cual deduce y toma nuevas decisiones.

Tienden a estar varios años atrasados con respecto a los avances en investigación de IA.

Los sistemas expertos se utilizan cuando NO existe un método capaz de encontrar la solución a un problema, pero se cuenta con un “experto” que puede colaborar con el desarrollo del sistema.

Para crear un SE, el conocimiento sobre el dominio del problema debe ser acotado y suficientemente estático. Los SE no se pueden aplicar en áreas en donde se necesite el sentido común, es decir, en dominios que no están acotados.

Componentes de los SE

Para funcionar, los SE necesitan:

- Acceder a una base de conocimiento (BC) sobre el dominio del problema
- Uno o más mecanismos de razonamiento
- Un mecanismo para explicar las decisiones que toman

El objetivo principal que nos propone el Sistema Experto es lograr que nuestro programa pueda realizar inferencias, o sea, teniendo un conjunto de conocimiento inicial, infiera acerca de situaciones presentadas para poder medir su resultado.

Para lograr esto tenemos que lograr que el conocimiento base (BC) tenga una forma **canónica** para entenderse, para que el sistema entienda las afirmaciones que se plantean sobre su ambiente.

A partir de la generación de una base de conocimiento canónica, logramos tener una base a la cual podemos agregar nuevo conocimiento y también preguntar sobre esta.

Ambas tareas requieren que realicemos inferencias, es decir, derivar nuevas sentencias de las antiguas. Es por esto que decimos, como indicamos anteriormente, que cada vez que el programa del agente es activado o utilizado, realiza dos cosas:

1. Le agrega a la BC lo que percibió (si no existe)
2. Le pregunta a la BC que acción debe ejecutar

Para saber que acción debe ejecutar debe razonar acerca de lo percibido, el estado del mundo actual, los efectos de las acciones posibles, etc.

Para lograr agregar el conocimiento inicial a la BC, es necesario utilizar la disciplina **Sintaxis y Semántica de los Lenguajes**.

Esta disciplina requiere sentencias bien formadas y le da un grado de verdad al mundo.

La implicación lógica de más importancia es:

$$a \mid = b$$

Si a es verdadera b también lo será

Es entonces que para buscar alguien que implique a tenemos que buscar en toda la base. La inferencia i es la que tenemos que buscar en la BC para ver si lo encontramos, quedaría entonces:

$$BC \mid = i \ a$$

a se deriva de la BC mediante i

Un algoritmo que deriva solo sentencias implicadas es sólido.

Un algoritmo de inferencia es **completo** si puede derivar cualquier sentencia que está implicada.

Si una BC es verdadera en el mundo real, entonces cualquier sentencia a que se derive de la BC, mediante un procedimiento de inferencia sólido, también será verdadera en el mundo real.

LÓGICA

La lógica es una disciplina que estudia métodos de **formalización del conocimiento** y formas válidas de **inferencia**.

Del Lenguaje Natural a la Lógica

El lenguaje natural se compone de diferentes tipos de frases: declarativas, imperativas e interrogativas. En el contexto de la lógica, las frases declarativas se transforman al **lenguaje de la lógica**, dando lugar a lo que se conoce como **fórmulas bien formadas** (fbf) o **forma normal**.

Niveles de Abstracción en Lógica

Existen diferentes niveles de abstracción en la lógica:

- **Lógica proposicional:** Toma como elemento básico las frases declarativas simples o proposiciones. Estas proposiciones constituyen una unidad de comunicación de conocimiento y pueden ser **verdaderas** o **falsas**. Por ejemplo, "Sócrates es humano" se representa como "SócratesHumano" en lógica proposicional. Sin embargo, la lógica proposicional no puede expresar relaciones más complejas, como "Todos los humanos son mortales" de manera generalizada, solo puede afirmarlo para cada individuo específico (ej. "SócratesMortal", "PlatónMortal").

La lógica proposicional define que cada verdad o falsedad es una afirmación completa y puede descomponerse en partes mas pequeñas.

SINTAXIS

Las sentencias atómicas se componen en un único símbolo proposicional y cada uno representa una proposición que puede ser verdadera o falsa. Cada una de estas se representa con letras mayúsculas (P,Q,R,etc.).

Las sentencias complejas se construyen con las simples (o atómicas) y usando conectores lógicos. Estos tienen que ver con el álgebra booleana por ende son clásicas y conocidas:

- $\sim$ (Negación)
- $\wedge$ (Conjunción) Es el AND
- $\vee$ (Disyuntor) Es el OR
- $\Rightarrow$ (Implica) Premisa $\Rightarrow$ Conclusión

SEMÁNTICA

La semántica en la lógica proposicional debe especificar cómo obtener el valor de verdad de cualquier sentencia, dado un modelo.

Este proceso se realiza de forma recursiva, todas las sentencias se construyen a partir de las sentencias atómicas y las cinco conectivas lógicas.

Para las sentencias complejas se utilizan tablas de verdad:

Tabla de verdad:

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$	$P \Leftrightarrow Q$
falso	falso	verdadero	falso	falso	verdadero	verdadero
falso	verdadero	verdadero	falso	verdadero	verdadero	falso
verdadero	falso	falso	falso	verdadero	falso	falso
verdadero	verdadero	falso	verdadero	verdadero	verdadero	verdadero

SATISFACTIBILIDAD

Una sentencia es satisfactoria si es verdadera para algún modelo. Si una sentencia a es verdadera en un modelo m entonces decimos que m satisface a , o que m es un modelo de a . La satisfactibilidad se puede averiguar enumerando los modelos posibles hasta que uno **satisface la sentencia**.

LÓGICA PROPOSICIONAL ENCADENAMIENTO HACIA ADELANTE Y HACIA ATRÁS

Las bases de conocimiento en aplicaciones del mundo real a menudo contienen un tipo específico de cláusulas que facilitan el razonamiento y la inferencia. Estas son las **Cláusulas de Horn**.

Cláusulas de Horn

Una **Cláusula de Horn** es una disyunción de literales, de los cuales, como máximo uno es positivo. Esta restricción es muy importante por varias razones:

1. **Cláusulas Positivas:** Las cláusulas de Horn con exactamente un literal positivo se denominan **cláusulas positivas**.
2. **Cabeza y Cuerpo:** El literal positivo se denomina **cabeza** y la disyunción de literales negativos se denomina **cuerpo** de la cláusula.
3. **Hechos:** Una cláusula positiva que no tiene literales negativos simplemente afirma una proposición dada, y se denomina **hecho**.

Forma de Implicación: Cada cláusula de Horn se puede escribir como una implicación cuya premisa sea una conjunción de literales positivos y cuya conclusión sea un único literal positivo.

• Ejemplo:

- Una cláusula de Horn como $\neg P \vee \neg Q \vee R$ (disyunción de literales, con un solo literal positivo R) puede reescribirse como una implicación: $(P \wedge Q) \rightarrow R$.
- Aquí, R es la cabeza (el literal positivo), y $P \wedge Q$ es el cuerpo (la conjunción de los literales negados en su forma positiva).
- Si no hay literales negativos, como en P , es un hecho. Esto se puede ver como $\text{True} \rightarrow P$, donde True es una conjunción vacía.

Importancia de las Cláusulas de Horn: La importancia de las Cláusulas de Horn radica en que la inferencia con ellas es computacionalmente más eficiente. Averiguar si hay o no implicación con

las cláusulas de Horn se puede realizar en un tiempo que es lineal respecto del tamaño de la base de conocimiento. Esto contrasta con la inferencia general en lógica proposicional, que puede ser NP-completa.

Inferencia con Cláusulas de Horn: Encadenamiento Hacia Adelante y Hacia Atrás

La inferencia con cláusulas de Horn se puede realizar mediante dos algoritmos principales:

1. Encadenamiento Hacia Adelante (Forward Chaining):

- El encadenamiento hacia adelante **parte de los datos conocidos** (los hechos).
- Procesa las reglas de la base de conocimiento para derivar nuevas conclusiones. Si las premisas de una regla son verdaderas (ya están en la base de conocimiento o se han inferido), entonces la conclusión de esa regla se añade a la base de conocimiento.
- Este proceso se repite hasta que no se puedan derivar nuevas conclusiones o se haya alcanzado el objetivo.
- Es un proceso "dirigido por los datos" o "data-driven". Es útil cuando se tienen muchos datos y se quiere ver qué conclusiones se pueden extraer.

2. Encadenamiento Hacia Atrás (Backward Chaining):

- El encadenamiento hacia atrás **parte desde una petición** (o un objetivo).
- Si se sabe que la petición es verdadera (es decir, ya es un hecho en la base de conocimiento), no es necesario realizar ningún trabajo.
- En caso contrario, el algoritmo encuentra aquellas implicaciones de la base de conocimiento que concluyen con la petición (es decir, la petición es la cabeza de la cláusula de Horn).
- Si se puede probar que todas las premisas (el cuerpo de la cláusula de Horn) son verdaderas, entonces la petición es verdadera.
- Este proceso se realiza de forma recursiva para cada una de las premisas que aún no son conocidas, convirtiéndolas en sub-peticiones.
- Es un proceso "dirigido por el objetivo" o "goal-driven". Es muy común en sistemas expertos para diagnóstico o consultas, donde se quiere probar una hipótesis específica.

Deducción por Negación (Reducción al Absurdo / Resolución):

- En este método, para probar que una petición P es verdadera, se **niega la petición** (se asume $\neg P$) y se añade a la base de conocimiento.
- Luego, se intenta encontrar una contradicción (la cláusula vacía $\square$) utilizando el proceso de resolución.
- Si se encuentra una contradicción, significa que la suposición inicial de que $\neg P$ era verdadera llevó a un absurdo, y por lo tanto, P debe ser verdadera.

- **Lógica de predicados:** En primer lugar, es importante entender que son los predicados, son funciones que describen propiedades o relaciones entre objetos.

Son algo más coloquial y de alto nivel comparado con las cláusulas. Sigue utilizando la notación booleana pero ahora las cosas las describiendo con funciones como si fuera código.

Utilizamos cuantificadores:

- **Cuantificador Universal ($\forall$):** Para todo
- **Cuantificador Existencial ($\exists$):** Existe al menos uno

Representa las frases declarativas con un mayor grado de detalle, considerando su estructura interna. Aquí, los elementos básicos son los objetos y las relaciones entre ellos. Se distingue qué se afirma (el predicado o relación) y sobre quién se lo afirma (el objeto). Por ejemplo, "Sócrates es humano" se representa como humano(Sócrates). Si un predicado tiene dos o más argumentos, indica una relación entre esos objetos, como odia(Marco, César).

- **Hechos y Reglas en Lógica de Predicados:** Si los argumentos de una fórmula son constantes (como "Marco"), la fórmula es un hecho. Si los argumentos son variables (como "x"), la fórmula es una regla.
- **Fórmulas conectadas:** Las fórmulas también pueden ser predicados conectados entre sí por conectivos, como $\text{leal}(x, \text{César}) \vee \text{odia}(x, \text{César})$. También pueden incluir implicaciones, como $\forall x: \text{romano}(x) \rightarrow \text{leal}(x, \text{César}) \vee \text{odia}(x, \text{César})$.

Las expresiones transformadas del lenguaje natural al lenguaje de la lógica se denominan fórmulas bien formadas (fbf)

FORMAS NORMALES

Las formas normales son estructuras estandarizadas para representar fórmulas lógicas, facilitando los procesos de inferencia.

- **Forma Normalizada Conjuntiva (FNC):** Es una conjunción de disyunciones. Un ejemplo es $(P \vee Q) \wedge (\neg P \vee R)$.
- **Forma Normalizada Disyuntiva (FND):** Es una disyunción de conjunciones. Un ejemplo es $(P \wedge Q) \vee (\neg P \wedge R)$.
- **Cláusulas de Horn:** Son una conjunción de disyunciones que no contienen más de un literal positivo (es decir, no más de una implicación). Un ejemplo es $(\neg P \vee \neg Q \vee R) \wedge (\neg T \vee P)$.

Conversión a Forma Normal/Clausal en Lógica Proposicional

Para aplicar el procedimiento de resolución, las fórmulas bien formadas (fbf) deben convertirse en un **conjunto de cláusulas**, donde cada cláusula es una fbf en FNC que no contiene ninguna conectiva de conjunción ($\wedge$). Por ejemplo, $(P \vee Q) \wedge (\neg P \vee R)$ se separa en dos cláusulas: $(P \vee Q)$ y $(\neg P \vee R)$.

El proceso de conversión a forma clausal sigue estos pasos:

1. **Eliminar las implicaciones:** Se usa la equivalencia $A \rightarrow B \equiv \neg A \vee B$.
 - Ejemplo: $\text{verano} \wedge \text{soleado} \rightarrow \text{calor}$ se convierte en $\neg(\text{verano} \wedge \text{soleado}) \vee \text{calor}$.

2. **Reducir el ámbito de las negaciones:** Se aplican las **leyes de De Morgan** y la **doble negación**.

- $\neg(\neg P) = P$
- $\neg(A \wedge B) = \neg A \vee \neg B$
- $\neg(A \vee B) = \neg A \wedge \neg B$
- Ejemplo: $\neg(\text{verano} \wedge \text{soleado}) \vee \text{calor}$ se convierte en $\neg \text{verano} \vee \neg \text{soleado} \vee \text{calor}$.

3. **Convertir a una conjunción de disyunciones:** Se utilizan las propiedades asociativa y distributiva, y al final, se separan en cláusulas eliminando las conjunciones.

- $(A \wedge B) \vee C = (A \vee C) \wedge (B \vee C)$
- $(A \vee B) \wedge C = (A \wedge C) \vee (B \wedge C)$
- $(A \vee B) \vee C = (A \vee B) \vee C, (A \wedge B) \wedge C = A \wedge (B \wedge C)$

MÉTODO DE RESOLUCIÓN – LÓGICA PROPOSICIONAL

El procedimiento para producir una demostración de una proposición P respecto de un conjunto de axiomas F es el siguiente:

1. Convertir todas las fbf de F a forma clausal/normal.
2. Negar P y convertirla a forma clausal/normal. Añadir las cláusulas obtenidas al conjunto.
3. Repetir hasta que se encuentre una contradicción o no se pueda seguir avanzando:
 - Seleccionar dos cláusulas y llamarlas **cláusulas padre**.
 - Calcular el **resolvente**.
 - Si el resolvente es la cláusula vacía ($\square$), se encontró una contradicción, lo que significa que **P es cierto**. Si no, **añadirlo al conjunto de cláusulas**.

Si se encuentra una contradicción, significa que lo que se quería probar era cierto. Si no se llega a la contradicción, no se pueden sacar conclusiones; **P puede ser cierto o falso**.

MÉTODO DE RESOLUCIÓN – LÓGICA DE PREDICADOS

El método es similar al utilizado en lógica proposicional, pero se le agrega un proceso adicional: la unificación.

Unificación

La unificación se produce utilizando sustituciones. El objetivo es que, en dos referencias a un mismo predicado, los términos lleguen a ser idénticos.

Ej.

-perro(x) $\vee$ carnívoro(x)

perro(Tobi)

$S[x/\text{Tobi}]$ – Sustituyo/unifico x por Tobi

-perro(Tobi) $\vee$ carnívoro(Tobi)

perro(Tobi)

Ej.: Lógica Proporcional

Demostrar $P \rightarrow (Q \wedge R)$

1. $\neg P \vee (Q \wedge R)$
2. $\neg P \vee Q \wedge \neg P \vee R$
3. Nos queda>
 - a. $\neg P \vee Q$
 - b. $\neg P \vee R$

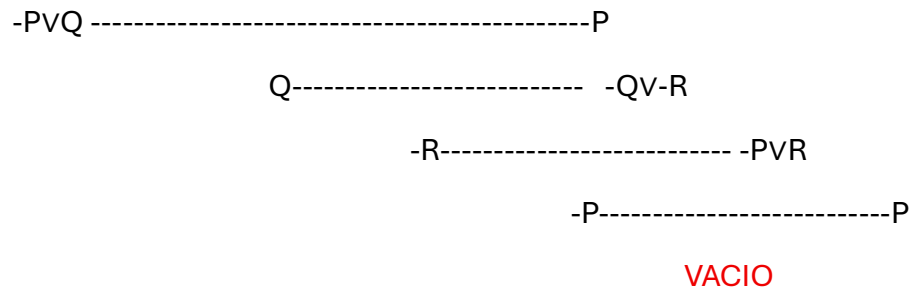
Negar la proposición a probar:

$$\neg (P \rightarrow (Q \vee R)) = P \vee \neg (Q \vee R)$$

Nos queda:

- P
- $\neg Q \vee \neg R$

Tomamos:



$P \rightarrow (Q \wedge R)$

$\neg P \vee Q$

$\neg P \vee R$

P

$\neg Q \vee \neg R$

MODELOS BAYESIANOS – UNIDAD 5

Razonamiento bajo incertidumbre

Cuando hablamos de incertidumbre **generalmente** nos referimos a situaciones donde el agente no tiene acceso a toda la verdad sobre su ambiente. Entonces se puede hablar de eventos que son verdaderos o falsos con cierta probabilidad.

Nos referimos entonces a situaciones en las que la información no es completamente cierta.

Puede involucrar:

- **Grados de veracidad:** Se refiere a lógica difusa, donde la verdad es una cuestión de grado y no a una dicotomía absoluta de verdadero o falso. Representa que tan probable o verdadera es un hecho o afirmación en un contexto específico. Se asocia a una medida numérica que indica la confianza en que un enunciado sea verdadero, generalmente en un rango de 0 a 1.
- **Grados de creencia:** Implica el uso de la **teoría de la probabilidad** para cuantificar la fuerza de la creencia en una proposición. Refleja el grado de confianza subjetiva que una persona o sistema tiene en una afirmación o hecho. No se basa únicamente en información objetiva, sino también en factores subjetivos, como experiencias previas o el contexto específico en que se evalúa la afirmación.

Los **Modelos Bayesianos** se construyen sobre la teoría de la probabilidad para representar y razonar sobre el conocimiento incierto.

Sistemas basados en la Teoría de la Probabilidad

La teoría de la probabilidad es una rama de las matemáticas que estudia la ocurrencia de eventos aleatorios y permite cuantificar la incertidumbre sobre estos eventos. Se utiliza para modelar y analizar situaciones en las que el resultado no es seguro, como en juegos de azar.

Para la resolución de los mismos necesitamos sistemas basados en la teoría de la probabilidad, que sean capaces de manejar incertidumbre y tomar decisiones en entornos inciertos.

Para manejar incertidumbre estos sistemas suelen tener lo que mencionamos como **grado de creencia** o **nivel de certeza** de los eventos. Desde esto se deben realizar inferencias probabilísticas sobre eventos futuros o desconocidos utilizando el histórico o base de conocimiento.

Los sistemas de este tipo se van actualizando cada vez que se obtiene un nuevo registro. Esto es para poder corregir la tasa de error siguiendo el **teorema de Bayes** dinámicamente:

$$P(A | B) = \frac{P(B | A)P(A)}{P(B)}$$

Estos sistemas implican:

- **Definición de un conjunto de variables:** Utilizadas para modelar el mundo
- **Tipos de variables:** Discretas/Continuas
- **Definición de los valores que puede tomar cada variable (dominio):** Por ejemplo, una variable como "Fiebre" o "Temperatura". Un ejemplo de valores es $\langle x_1, x_2, x_3 \rangle$, o "cierto, falso".

TEORÍA DE LA PROBABILIDAD

Probabilidad a priori o incondicional: Es el grado de creencia que se le otorga a una proposición en ausencia de cualquier otra información. A priori es como antes de empezar a hacer nada, es una probabilidad “estimada” que tenemos del evento.

Ej.: $P(X=x1)=0.3$ o $P(x1)=0.3$; $P(\text{Test}=positivo)=0.3$ o $P(positivo)=0.3$.

Probabilidad a posteriori o condicional: Es la probabilidad de que ocurra un evento **a** dado que ocurre un evento **b** (evidencia).

No implica causa efecto.

Como el nombre indica, viene después, se establece después de realizar todos los cálculos y darle una evidencia más determinante.

Ej.: $P(X=x1|Y=y1)=0.7$ o $P(y1)=0.7$;

$P(\text{Test} = \text{positivo} | \text{Enfermedad} = \text{presente})=0.7$ o $P(\text{positivo presente})=0.7$.

La fórmula para la probabilidad condicional es $P(a | b) = \frac{P(a \wedge b)}{P(b)}$, la cual puede reordenarse a

$$P(a \wedge b) = P(a | b)P(b).$$

Cuando trabajamos con probabilidades condicionales, la regla se extiende de manera similar. Si ya estamos condicionando todo a un evento C, la regla de la cadena se aplica de esta forma:

$$P(a, b | c) = P(a | b, c) * P(b | c)$$

Ej.:

En el hospital se estudia la probabilidad de que a una persona le toque una revisión completa por un doctor. Se utilizan ciertos síntomas para determinar si tiene algún tipo de enfermedad y conlleva a estudios más concretos:

- El 2% de las personas tiene algún tipo de enfermedad de este tipo
- Cuando una persona tiene una enfermedad el 80% experimenta un síntoma específico
- Cuando alguien no tiene nada existe un 5% de probabilidad de que siga experimentando síntomas. Tasa de falsos positivos.

PASO A PASO

1. Definir los eventos

E: La persona tiene la enfermedad (E = Presente)

S: La persona presenta los síntomas específicos (S=Positivo)

La probabilidad la definimos como: $P(E|S)$

2. Extraer probabilidades dadas

$P(E) = 0,02$

$P(-E) = 0.98$

$$P(S|E) = 0,8$$

$$P(S|\neg E) = 0,05$$

3. Calcular la probabilidad total de presentar síntomas

Para calcular la probabilidad de que una persona tenga una enfermedad dados los síntomas, debemos:

$P(S)$ = Probabilidad total de presentar síntomas

Este calculo se hace usando la ley de probabilidad total

la **Ley de Probabilidad Total** establece que la probabilidad del evento A se puede calcular como:

$$P(A) = \sum_{i=1}^n P(A | B_i)P(B_i)$$

O, de manera expandida:

$$P(A) = P(A | B_1)P(B_1) + P(A | B_2)P(B_2) + \dots + P(A | B_n)P(B_n)$$

Resolvemos:

$$P(S) = P(S | E) \cdot P(E) + P(S | \neg E) \cdot P(\neg E)$$

$$P(S) = 0.8 \cdot 0.02 + 0.05 \cdot 0.98$$

$$P(S) = 0.065$$

4. Calculamos $P(E|S)$ usando la Regla de Bayes

$$P(E | S) = \frac{P(S | E) \cdot P(E)}{P(S)}$$

$$P(E|S) = 0.8 \cdot 0.02 / 0.065$$

$$P(E|S) = 0.246$$

La probabilidad de que una persona **tenga la enfermedad dado que experimenta el síntoma** es aproximadamente del **24.6%**.

Esto significa que, si una persona presenta el síntoma, la probabilidad de que realmente tenga la enfermedad aumenta significativamente (del 2% inicial al casi 25%). Esto justificaría la "revisión completa por un doctor" y "estudios más concretos" mencionados en el problema, ya que el síntoma es un buen indicador, aunque no perfecto.

Independencia de eventos

Dos eventos a y b son independientes si $P(a|b) = P(a)$

Esto también implica $P(a \wedge b) = P(a)P(b)$.

La dependencia no implica causalidad.

Independencia condicional:

Los eventos a y b son **condicionalmente independientes** dado c si $P(a|b \wedge c) = P(a|c)$.

Esta propiedad es muy importante para la representación de la información en las redes bayesianas.

REDES BAYESIANAS

Las **redes bayesianas** son modelos probabilísticos que representan relaciones de dependencia entre un conjunto de variables mediante un gráfico acíclico dirigido. Cada nodo en la red representa una variable y las aristas entre los nodos indican relaciones de dependencia probabilística entre ellas.

Las redes bayesianas se utilizan para realizar inferencias, diagnosticar problemas y tomar decisiones en condiciones de incertidumbre.

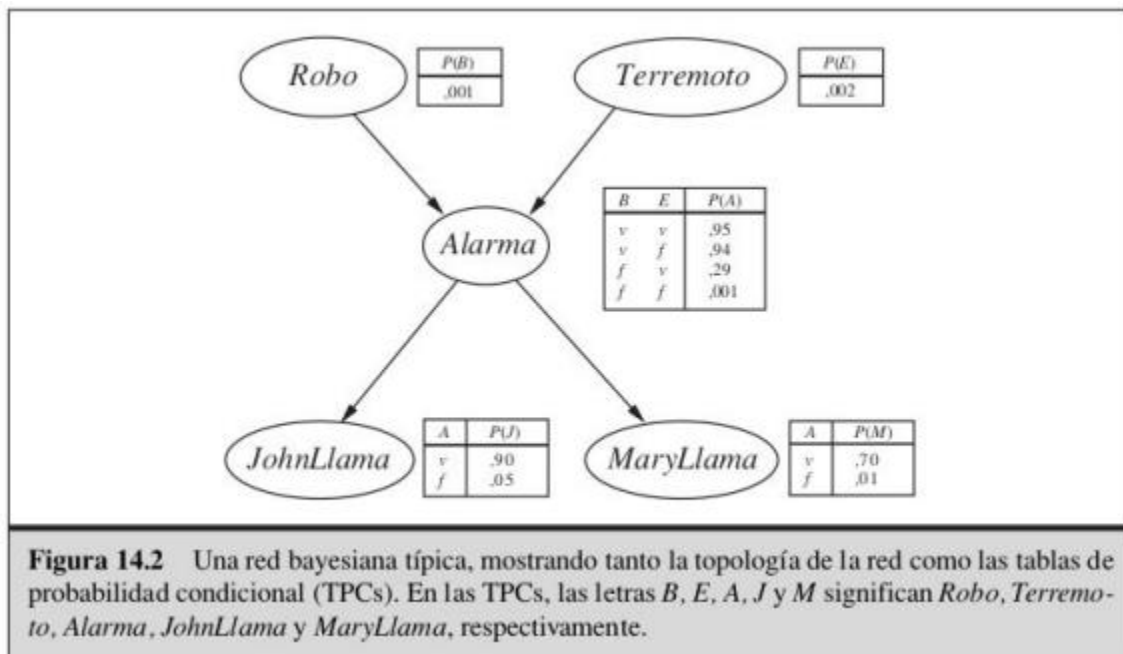
Muestran una descripción compacta de cualquier distribución de probabilidad conjunta completa.

Son grafos dirigidos, y cada nodo contiene información probabilística.

La especificación completa incluye:

- Un conjunto de variables aleatorias que forman los nodos.
- Un conjunto de arcos dirigidos que conectan pares de nodos; si hay un arco de X a Y , X es padre de Y .
- Cada nodo X_i tiene una distribución de probabilidad condicionada $P(X_i | \text{Padres}(X_i))$.
- El grafo no tiene ciclos dirigidos, es un grafo acíclico dirigido (GAD).

El significado intuitivo de un arco de X a Y es que X tiene una influencia directa sobre Y .



Inferencia en Redes Bayesianas

El objetivo es calcular la distribución de probabilidad a posteriori para un conjunto de variables pregunta, dado algún evento observado.

- **Variables involucradas:** Existen tres tipos de variables aplicadas en los nodos en todo momento
 - **VARIABLES OBSERVADAS/EVIDENCIAS:** Son las variables de las que se tiene información o evidencia en el momento de realizar la inferencia. Ayudan a actualizar las probabilidades en la red.
 - **VARIABLES DE CONSULTA/PREGUNTA:** Son las variables sobre las que queremos obtener información o calcular su probabilidad condicional dado el conjunto de evidencias.
 - **VARIABLES LATENTES/OCULTAS:** Son variables que no observamos directamente ni estamos consultando, pero que pueden influir en el cálculo de la probabilidad de las variables de consulta. Estas variables pueden afectar las dependencias en la red y deben tenerse en cuenta en los cálculos.

La fórmula general para la inferencia es $P(X|e)$, donde X es la variable de consulta y e es la evidencia.

Métodos de Inferencia:

- **Usando las distribuciones conjuntas totales:** Se identifican los sucesos atómicos en los que la proposición se cumple y se suman sus probabilidades. **No es una inferencia en una red bayesiana y no es eficiente.**
- **Con la REGLA DE BAYES:**
 - Mediante un proceso de razonamiento "artesanal".
 - Se usa el ejemplo del robo para demostrar esto.
 - La pregunta es $P(r|j \wedge m)$
 - La respuesta se calcula como

$$P(r|j \wedge m) = \frac{P(j \wedge m|r)P(r)}{P(j \wedge m)}$$

Inferencia automática:

- **Exacta:**
 - **Por enumeración**

Una probabilidad conjunta se puede calcular como un producto de probabilidades condicionales:

$$\mathbf{P}(x_1, \dots, x_n) = \prod_{i=1}^n P(x_i | \text{padres}(X_i))$$

Sumando las probabilidades conjuntas de los casos donde se cumplen las proposiciones se puede calcular la probabilidad buscada.

$$\mathbf{P}(X|e) = \alpha \mathbf{P}(X, e) = \alpha \sum_y \mathbf{P}(X, e, y)$$

(Esto este re raro no lo entiendo)

- **Aproximada**

Consideraciones para Redes Grandes

Llevar a cabo un proceso de deducción y análisis similar al de los ejemplos anteriores en redes de gran tamaño es prácticamente imposible. La representación de las probabilidades conjuntas totales tendría un tamaño inmanejable.

Los mecanismos de inferencia automáticos deben enfocarse en la eficiencia. No es posible crear estas redes de forma manual, ya que el conocimiento del dominio para definir la topología y las probabilidades conocidas no son suficientes. En estos casos, se deben utilizar métodos automáticos para "aprender" las redes.

Para que tenga sentido que un agente inteligente utilice una red bayesiana, el ambiente debe cumplir con algunas condiciones:

1. **Incertidumbre:** El ambiente debe tener un grado de incertidumbre en la información disponible. Las redes bayesianas son especialmente útiles en contextos donde hay probabilidad y se requiere inferencia basada en evidencia parcial
2. **Dependencias condicionales claras:** Debe haber relaciones condicionales claras entre las variables, ya que las redes bayesianas modelan dependencias probabilísticas entre ellas.
3. **Disponibilidad de datos históricos:** Es ideal que exista un conjunto de datos que permita definir las probabilidades condicionales de las variables. Si el agente tiene acceso a datos históricos o a un conocimiento previo que permita definir las probabilidades, entonces la red bayesiana será más precisa.
4. **Necesidad de inferencia continua:** Cuando un agente necesita actualizar sus creencias a medida que recibe nueva información o evidencia, una red bayesiana facilita este proceso de manera eficiente.

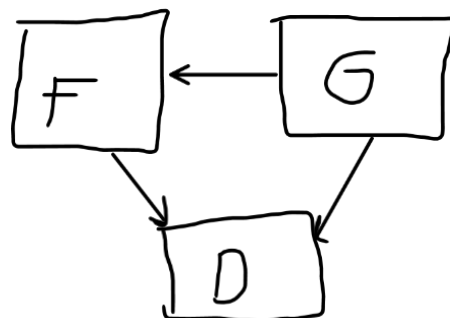
Ej.:

Supongamos una red bayesiana con 3 variables:

F: Fiebre (Tiene – No tiene)

G: Gripe (Tiene – No tiene)

D: Dolor de cabeza (Tiene – No tiene)



La red bayesiana tiene las siguientes dependencias:

- Tener gripe (G) influye en tener fiebre (F).

- Tener fiebre y gripe también afecta a tener dolor de cabeza.(D)

Probabilidades:

- $P(G) = 0.1$
- $P(-G) = 0.9$
- Probabilidad de tener fiebre dado que se tiene gripe: $P(F|G) = 0.8$
- Probabilidad de tener fiebre sin tener gripe: $P(F|-G) = 0.2$
- Probabilidad de tener dolor de cabeza dado fiebre y gripe: $P(D|F,G)$
 - o $P(D|F,G) = 0.9$
 - o $P(D|F,-G) = 0.6$
 - o $P(D|-F,G) = 0.4$
 - o $P(D|-F,-G) = 0.1$

Definición de Fiebre y Gripe:

Lo que queremos calcular es: La probabilidad de que una persona tenga gripe dado que tuvo fiebre y dolor de cabeza antes. $P(G|F,D)$

Se resuelve por Bayes:

$$P(G|F,D) = P(F,D|G) * P(G) / P(F,D)$$

- o Calcular $P(D,F|G)$, por la **regla de la cadena condicional**, podemos calcular:

$$\begin{aligned} &P(D|F,G) * P(F|G) \\ &0.9 * 0.8 \\ &\mathbf{0.72} \end{aligned}$$

- o Calcular $P(F,D) = P(F,D|G) * P(G) + P(F,D|-G) * P(-G)$
 $P(D|F,-G) * P(F|-G) = 0.6 * 0.2 = 0.12$

$$P(F,D) = 0.72 * 0.1 + 0.12 * 0.9 = \mathbf{0.18}$$

- o $P(G|F,D) = P(F,D|G) * P(G) / P(F,D)$
 $P(G|F,D) = 0.72 * 0.1 / 0.18 = \mathbf{0.4}$

La probabilidad de que un paciente tenga gripe dado que tiene fiebre y dolor de cabeza es del 0,4 (osea 40% aproximadamente).